BaitFL: Lightweight Trap-Based Free-Rider Detection and Mitigation in Federated Learning

Aashnan Rahman

***Abstract*—Free riders obtain the shared federated model without performing local training, while fabricated responses can resemble genuine updates. BaitFL audits this behavior through undisclosed model challenges, update norms and layer-level profiles. Geometric tests and repeated fresh evidence govern removal, while bounded review limits exclusion when the evidence remains inconclusive. The server requires neither an auxiliary detector nor client-reported loss or attack identities. Experiments across three random seeds cover MNIST and CIFAR-10, IID and non-IID partitions, four non-adaptive fabrication strategies, and 30% and 40% free riders. All free riders in this main grid are removed, giving 100.00% recall across 96 runs. There are 3 honest removals, concentrated in 2 runs. Additional frozen-setting checks found no honest removal in four honest-only runs, but FR2–FR4 were undetected at 5% prevalence in the completed non-IID tests. These results identify a lower-prevalence limitation. An oracle norm-matching feature diagnostic also substantially reduces first-cycle certification, exposing a limit to magnitude-based separation. In separately timed 100-round runs, server-side checking accounted for 0.58% of total computation time on MNIST and 0.89% on CIFAR-10. BaitFL is lightweight in server computation for the evaluated population of 100 clients. Even IID settings withhold about 11% of honest responses through auditing. Withheld contributions and honest-client errors remain costs of the method, which requires visibility of individual client responses.**

***Index Terms*—federated learning, free-rider detection, hidden challenges, robust statistics, non-IID data, client auditing**

I. Introduction

Federated learning (FL) allows multiple clients to train a shared model without handing over their local data. In each communication round, the server distributes the current global model, the clients update it using their own data, and the server aggregates the returned updates into a revised model [1]. Since raw data stays with its owner, participants can collaborate without first creating a central repository of their records. Keeping data local does not remove every privacy risk, as shared updates can still reveal information about training data [2]–[4]. Exchanging updates instead of datasets also avoids moving large volumes of local data, although repeated model exchange remains a significant communication cost in FL [1]. Effective collaboration depends on clients performing useful local training, even though the server cannot directly observe that work.

Like other distributed systems, FL is open to misconduct. Label-flipping and model-poisoning attacks, for example, alter local data or submitted updates in order to degrade the global model or introduce targeted behavior [2], [5]. Free riding is a quieter problem. The participant's primary aim is to obtain the shared model without paying the cost of local training [6], [7]. A client may lack useful data or computational resources, or may simply choose to conserve them. Instead of training, it can return an unchanged model, fabricate a noisy response, or build an update from models received in earlier rounds. Such responses may resemble what the server expects, so they need not visibly disrupt learning and can evade checks aimed at overtly harmful updates. Free riding shifts the training burden to honest participants. When a client withholds useful data and computation, the federation also loses a contribution that could have improved the shared model. Figure 1 shows the standard model exchange. All clients receive the global model. Honest clients train it on local data and return their local models, while a free rider submits a fabricated model without local optimization.

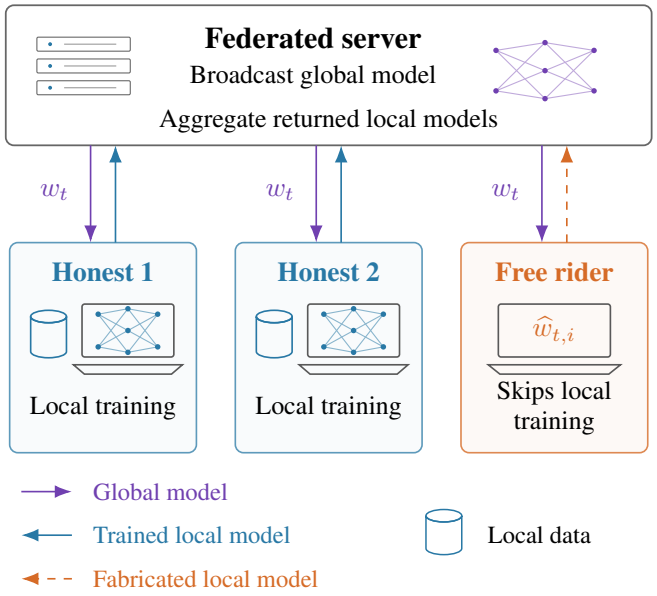


Fig. 1. Free riding in federated learning. The server broadcasts the global model. Honest clients return locally trained models, while a free rider returns a fabricated model without local training.

Detecting this omission is difficult because unusual updates are not limited to free riders. Honest clients differ in sample sizes, class distributions, and local learning trajectories, particularly under non-IID data [2]. A small update can therefore reflect legitimate training, while a fabricated one can look like an ordinary contribution. The server has to tell these cases apart without observing local optimization. Existing approaches rely on learned anomaly models, parameter audits, or evidence obtained through inference attacks [6], [8], [9]. This raises a practical question: can lightweight server-side checks detect

Aashnan Rahman is with the Department of Computer Science and Engineering, Islamic University of Technology.

free riding without training an auxiliary detector? There is also a cost to being wrong. Withholding an honest client's updates can reduce its contribution to the shared model, even if it is never permanently removed. A detector must therefore consider both the evidence needed for removal and the duration of temporary exclusion.

To address this problem, we propose *BaitFL*, a lightweight trap-based method for detecting and mitigating free riders in federated learning. In the 96-run main grid at 30% and 40% prevalence, it removes all tested free riders, with 3 honest removals in 2 runs. Checking accounts for less than 1% of computation in the instrumented executions. Additional low-prevalence and norm-matching checks expose limits to this result rather than supporting prevalence-independent detection. The main contributions of this work are as follows:

- **Lightweight server-side detection.** We develop a challenge-based detector using update norms and tensor profiles, without training an auxiliary detection model or accessing clients' private training data.
- **Confirmation using repeated evidence.** We design a decision mechanism that separates initial suspicion from permanent removal and requires fresh evidence of the same component across two challenges and distinct audit cycles for statistical confirmation.
- **Bounded mitigation and rehabilitation.** We retain strong evidence across ambiguous cycles while allowing unconfirmed quarantine to expire and clearing statistical memory after repeated clean observations.
- **Evaluation across four free-rider behaviors.** We study unchanged-model responses, random fabrication, history averaging, and trajectory imitation on MNIST and CIFAR-10 under IID and non-IID partitions, with 30% and 40% free riders.

Sections II and III review related work and define the threat model. Section IV presents the auditing procedure, followed by results in Section V and complexity analysis in Section VI. Section VII discusses security, limitations and future work before the conclusion.

II. Literature Review

Free-rider detection concerns omitted training, whereas poisoning defenses primarily address harmful updates. Lin *et al.* introduced fabrication attacks and STD-DAGMM, combining update statistics with an autoencoder and mixture model [6], [10]. Fraboni *et al.* studied unchanged-model and disguised, decaying-noise responses in model aggregation [7]. Zhu *et al.* developed scaled-delta and advanced attacks that imitate training trajectories [11]. These studies show why an attacker need not submit an obviously damaging response. They also motivate testing magnitude-matched fabrication rather than assuming every free rider produces a small update.

Learned detectors and contribution audits obtain different kinds of evidence. STD-DAGMM requires auxiliary fitting, whose cost depends on architecture and training budget. FRAD combines learned detection with contribution estimates and reputation [12]. Its additional context depends on reliable resource and contribution information. PASS asks clients to evaluate peers' models on local data and report contribution scores [8]. It avoids central access to raw data, but adds peer evaluation and exposes models to auditors. Low utility on an evaluator's data can also reflect heterogeneity rather than omitted training.

FRIDA uses membership and property inference as training evidence [9]. Its membership variants introduce canaries and client training work, while property variants infer local dataset characteristics. These are distinct information and computation tradeoffs, not direct inspection of raw records. BaitFL likewise has a privacy limit: attributable responses must remain visible. It cannot operate unchanged behind secure aggregation [13], and keeping raw data local does not prevent leakage through updates [3], [4].

Robust aggregation offers another point of comparison. Krum, trimmed aggregation and geometric-median methods reduce the effect of Byzantine updates [14]–[16]. FLTrust uses a trusted server dataset [17]. These methods aim to protect learning under their assumptions; rejecting a harmful update does not establish whether its sender trained. FedProx, SCAFFOLD and FedNova address heterogeneous local optimization [18]–[20]. They can improve honest training but do not, by themselves, audit a participant that skips the prescribed computation.

Proof-of-learning and VeriML seek evidence about executed training or outsourced learning computations [21], [22]. They provide a different assurance from population-relative anomaly scores and require their own verification artifacts. Data-Shapley and federated valuation estimate marginal usefulness [23]–[25]; usefulness remains different from evidence of honest participation.

BaitFL tests responses to altered starting models using compact statistics and repeated evidence. Our comparisons cover different endpoints: no-defense aggregation for utility, a declared STD-DAGMM implementation for timing and flag diagnostics, and published information requirements qualitatively. They do not constitute a common-workload ranking against PASS, FRAD or FRIDA. Supplementary Table S1 retains the detailed requirements comparison.

III. Threat Model

A. Participants and Server Knowledge

A trusted server coordinates N clients with persistent identities and reliable communication. Every active client responds in each round. The population is fixed, with no client sampling, late arrivals or dropouts. Removal ends participation. Quarantined clients still return responses, but those responses are excluded from aggregation, as are all responses to traps. Honest clients train the model they receive with the prescribed optimizer. Their datasets may differ in size and class distribution, covering both IID and non-IID partitions.

The server may send different models to different clients without disclosing which are traps. For client i in round t, it knows the sent model, $w_{t,i}^{\mathrm{send}}$, and the returned model, $w_{t,i}^{\mathrm{ret}}$. These give the update and its norm,

$$\Delta_{t,i} = w_{t,i}^{\mathrm{ret}} - w_{t,i}^{\mathrm{send}}, \qquad r_{t,i} = \|\Delta_{t,i}\|_2. \tag{1}$$

Detection uses these responses and the server's audit history. The server has no access to private training records and no prior knowledge of which clients are free riders or how they

construct their responses. Individual responses are visible to it, so the setting does not use secure aggregation.

B. Free-Rider Objective and Scope

Free riders seek the shared model while avoiding local training costs. Some lack suitable data or training resources. Others have both but choose to withhold them. They can still receive and store models and carry out the relatively simple calculations needed to fabricate a response. Introducing a targeted error or backdoor is outside their objective here.

Our experiments consider persistent free riding without collusion, following Lin *et al.* [6]. These clients perform no local optimization at any stage. Partial training and behavior that alternates between training and free riding fall outside this scope. Each attacker relies only on its own received-model history and local randomness. They do not share responses or change identities. The evaluated strategies do not attempt to recognize traps or adapt to evade audits. Section VII-C discusses those possibilities.

We assume an honest majority because screening uses peer statistics and seeks a separated minority of suspicious responses. The experiments use 30% and 40% free riders, with one fixed fabrication strategy shared by the free riders in each run.

C. Evaluated Free-Rider Behaviors

Let $\widehat{w}_{t,i}$ denote the model returned by a free rider. All four strategies below skip local optimization.

FR1: current-model sendback.

Returning the received model unchanged gives a zero update, whether the model is ordinary or a trap:

$$\widehat{w}_{t,i} = w_{t,i}^{\mathrm{send}}. \tag{2}$$

FR2: bounded random fabrication.

Independent uniform noise is added to each floating point entry of the received model state:

$$\widehat{w}_{t,i} = w_{t,i}^{\mathrm{send}} + \epsilon_{t,i}, \quad (\epsilon_{t,i})_j \sim \mathcal{U}(-10^{-3}, 10^{-3}). \tag{3}$$

FR3: received-history averaging.

With a history length H, the response averages the most recent $\min(H, t)$ received model states, stored in $\mathcal{H}_{t,i}$:

$$\widehat{w}_{t,i} = \frac{1}{|\mathcal{H}_{t,i}|} \sum_{w \in \mathcal{H}_{t,i}} w. \tag{4}$$

This history includes the current model and any earlier traps. The client therefore follows recent model evolution without training on local data.

FR4: received-trajectory imitation.

Changes between consecutive received models provide an estimated update. Let $\overline{d}_{t,i}$ average up to the five most recent differences:

$$d_{s,i} = w_{s,i}^{\mathrm{send}} - w_{s-1,i}^{\mathrm{send}}, \tag{5}$$

$$\widehat{w}_{t,i} = w_{t,i}^{\mathrm{send}} + \overline{d}_{t,i} + \eta_{t,i}. \tag{6}$$

For a standard Gaussian vector $z_{t,i}$, the noise is scaled as

$$\eta_{t,i} = 0.1 \, \frac{\max(\|\overline{d}_{t,i}\|_2, \epsilon_0)}{\max(\|z_{t,i}\|_2, \epsilon_0)} \, z_{t,i}, \qquad \epsilon_0 = 10^{-6}. \tag{7}$$

Equation (7) applies to vectors covering all floating model entries. On the first response, $\overline{d}_{1,i} = 0$ and $\eta_{1,i}$ instead has independent Gaussian entries with standard deviation 10^{-4}. Traps enter the same history as ordinary models and affect the estimated trajectory.

IV. Proposed Methodology

BaitFL compares clients through hidden model challenges, then separates initial suspicion from the evidence needed for removal. Figure 2 shows the round schedule, Figure 3 shows the challenge mechanism, and Figure 4 summarizes client participation states. Warmup establishes an ordinary training trajectory (Phase I). Paired coverage collects comparable responses (Phase II), and confirmation updates the client records (Phase III). Outside paired cycles, ordinary training continues with sparse audits (Phase IV).

The server first runs warmup and initial paired coverage. Thereafter, it repeats paired cycles while any client has an active review budget or rehabilitation probation. Otherwise, it executes ordinary rounds and any scheduled sparse audit until the training horizon T is reached. The continuation rules appear at the bottom of Figure 2. The illustrated second paired cycle permits earliest removal at round 30. The example then enters ordinary training with audits after ten quiet rounds. The two challenge labels summarize exposures within each paired cycle. The records $\mathcal{S}$ retain client status, quarantine, review budget, component counts, clean progress and cooldown. Counts start at zero and the quiet clock is initially unset, $\sigma = \perp$. The study uses warmup length $T_{\mathrm{warm}} = 10$, paired-cycle length $\ell_{\mathrm{cyc}} = 10$ and sparse-audit interval $I = 10$.

The challenge changes a client's starting point without changing the requested training task. Honest optimization responds to that starting point through the local data. An unchanged response, arbitrary noise or recycled model history need not respond in the same way. Comparing clients that received the same challenge can expose differences in the size and layer distribution of their updates. Two independent challenges test whether the difference recurs. This is behavioral evidence rather than proof of training, since fabricated and genuine responses may still overlap.

A. Challenges, Measurements, and Aggregation

Each paired cycle starts from a frozen copy $w^{(c)}$ of the current global model. For floating tensor k, the server creates traps A and B as

$$\tau_{a,k} = w_k^{(c)} + \gamma \max\{\mathrm{sd}(w_k^{(c)}), \epsilon_\tau\} \xi_{a,k}, \quad a \in \{A, B\}. \tag{8}$$

In (8), $\xi_{a,k} \sim \mathcal{N}(0, I)$ uses independent draws, and sd is the tensor's population standard deviation. The study sets $\gamma = 0.01$ and $\epsilon_\tau = 10^{-4}$. Floating buffers, including BatchNorm statistics, are perturbed along with weights. Integer counters remain unchanged. The two traps stay fixed within the cycle, even as ordinary responses update the global model.

TABLE I
PRINCIPAL NOTATION AND FIXED PROTOCOL SETTINGS.

Symbol	Meaning	Study setting
N, T	Initial client population and training horizon	100 clients, 100 rounds
w_t, τ_A, τ_B	Global model and the two shared challenges	Updated globally, fixed per cycle
Δ_i^a, r_i^a, p_i^a	Response change, its norm and its tensor profile	Measured from returned models
$z_i^{n,a}, z_i^{p,a}$	Leave-one-out norm and profile scores	Population-derived
$T_{\text{warm}}, \ell_{\text{cyc}}, I$	Warmup, paired-cycle length and sparse-audit interval	10, 10 and 10 rounds
γ, ϵ_τ	Relative challenge strength and tensor-scale floor	0.01, 10^{-4}
B	Bootstrap replicates	1,000
P, L	Floating model coordinates and floating tensors	Architecture-dependent
$\mathcal{U}, \mathcal{V}$	Nominated cohort and remaining active peers	Statistical certification sets
$\mathcal{S}, \mathcal{E}$	Client records and paired cycle decisions	Retained state and fresh evidence
strong, unresolved, clear	Cycle outcomes	Distinct from retained client status
$\mathcal{F}, \mathcal{D}, \mathcal{F}_{\text{elig}}$	Sparse findings, cooldown clients and eligible findings	$\mathcal{F}_{\text{elig}} = \mathcal{F} \setminus \mathcal{D}$
$\mathcal{W}_{\text{det}}$	Total detector work over the training horizon	Bound in Section VI

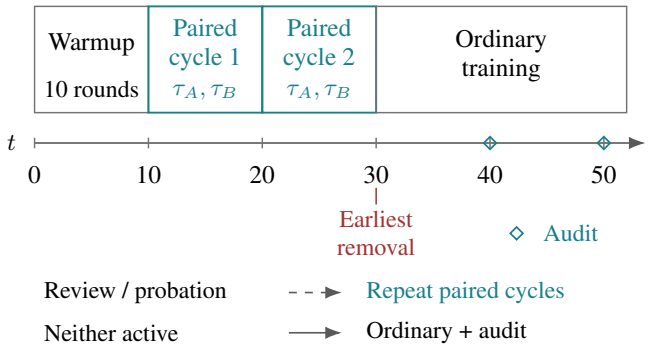


Fig. 2. Illustrative round schedule from warmup and paired coverage to ordinary training with sparse audits.

For a response to trap a, write $\Delta_{i,k}^a$ for the returned tensor minus the tensor sent to client i. Its norm r_i^a measures total update magnitude. The scale-free profile in (9) is

$$p_{i,k}^a = \frac{\|\Delta_{i,k}^a\|_2}{\sum_j \|\Delta_{i,j}^a\|_2}. \tag{9}$$

The profile is zero when its denominator is below 10^{-6}. Otherwise, it records where the update magnitude lies across floating tensors.

For each trap, client i is compared with all other active clients having a selected response to that same trap. Quarantined and unresolved clients are not removed from this reference. Let m_i^a and d_i^a be the median and median absolute deviation (MAD) of the peer norms. Then

$$z_i^{n,a} = \frac{r_i^a - m_i^a}{d_i^a / \Phi^{-1}(0.75)}, \tag{10}$$

where Φ is the standard normal cumulative distribution function. The factor normalizes the MAD. It does not assume that client updates are normally distributed.

The profile reference $\widetilde{p}_{-i}^a$ is the coordinatewise median of the peer profiles. The implementation uses the lower middle value when a tensor coordinate has an even number of peer observations. Define the client's profile distance as $\rho_i^a = \|p_i^a - \widetilde{p}_{-i}^a\|_2$. Peer distances are computed against that same reference. Their median μ_i^a and MAD δ_i^a give

$$z_i^{p,a} = \frac{\rho_i^a - \mu_i^a}{\delta_i^a / \Phi^{-1}(0.75)}. \tag{11}$$

A missing comparison or zero MAD leaves the corresponding score undefined.

Send-back is checked separately. It means an exactly zero floating-point update norm, or a returned model whose fingerprint matches the immediately previous model sent to that client. The latter comparison covers every tensor in the model state. The server retains one previous-sent fingerprint per client, not a history of full returned models.

The representative response geometry in Figure 3b uses the seed-42 execution. The plotted averages summarize the two challenges and do not replace the paired component certificates.

Aggregation runs alongside these checks. Let $\mathcal{A}_t$ contain clients that received the ordinary model and were not quarantined at the start of round t, excluding any newly quarantined or removed at its end. Then

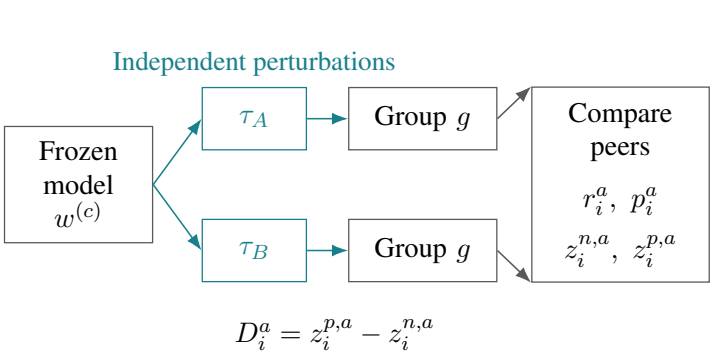
$$w_{t+1} = w_t + \frac{1}{|\mathcal{A}_t|} \sum_{i \in \mathcal{A}_t} \Delta_{t,i}. \tag{12}$$

Equation (12) applies to floating tensors with equal client weights. This gives each eligible client equal influence, unlike sample-size weighting in the usual FedAvg formulation [1]. The server does not need client sample counts. Both the detector study and its no-defense comparison use this same weighting, so their utility differences are evaluated under a common aggregation objective. Non-floating entries retain their server values. An empty $\mathcal{A}_t$ leaves the model unchanged. Training on a trap replaces ordinary local training in that round. The trap response never enters aggregation.

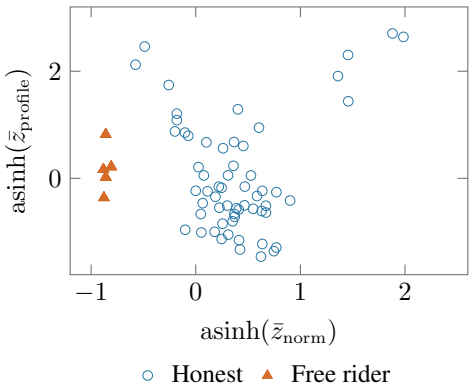
B. Phases I and II: Warmup and Paired Coverage

During the first $T_{\text{warm}} = 10$ rounds, all active clients receive the ordinary model and aggregate under (12). Measurements and previous-sent fingerprints are recorded, but no client is removed. In the next cycle, a random partition into at most ten groups gives every client one selected response to each challenge. The selected positions are fixed before responses are observed. Challenge recipients do the requested local training, but their responses are withheld from aggregation. Ordinary non-quarantined responses continue to contribute.

After initial coverage, reviewed and other clients are grouped separately. Reviewed clients alternate challenges throughout the cycle, while the other clients receive one selected exposure to each. Only the preselected pair enters the cycle decision.



(a) Challenge construction and comparison. Each group receives both perturbed models, and the server compares norms and tensor profiles among clients receiving the same trap.



(b) Recorded response geometry. CIFAR-10, non-IID data, FR3, 40% free riders, round 20. Axes show transformed paired mean scores.

Fig. 3. Model challenges and representative client responses. Point shapes identify evaluation labels unavailable to the detector. Panel (b) contains 40 free riders, but coincident score pairs leave only about four distinct triangle locations visible.

Missing A/B coverage cannot confirm removal. Screening and state changes occur before the last aggregation, so newly quarantined or removed responses are excluded. Algorithm 1 connects these phases to ordinary training and sparse auditing. Supplementary Algorithms S1–S4 give the expanded procedures for the published study engine.

1) Screening: For each client, form $D_i^A = z_i^{p,A} - z_i^{n,A}$, $D_i^B = z_i^{p,B} - z_i^{n,B}$ and their average. Each view uses $\lceil\sqrt{n_v}\rceil$ equally spaced histogram bins, where n_v counts clients with valid paired scores. Otsu's criterion selects a split [26]. The lower side of the average-view split must contain a majority of all active clients, not merely a majority of the valid-score subset.

The server resamples clients jointly across the three views in 1,000 bootstrap replicates, obtaining a 99% interval for each split boundary [27]. A client above every upper boundary is nominated as suspicious. One below every lower boundary is clear. Other outcomes remain unresolved. These intervals describe split stability, not a calibrated false-removal probability.

If the population split is unavailable or unstable, negative norm-z scores on both traps nominate suspicion. Here zero denotes the measured peer median. Neither score negative gives a clear result. One negative score or an undefined comparison leaves the result unresolved. Paired send-back evidence is handled independently of this statistical fallback.

2) Geometric evidence: A statistical nomination must pass a separate separation test. Let $\mathcal{U}$ be the nominated cohort and $\mathcal{V}$ the remaining active clients. With n active clients, statistical certification requires

$$\left\lceil \frac{n}{\lceil\sqrt{n}\rceil} \right\rceil \le |\mathcal{U}| < |\mathcal{V}|. \tag{13}$$

The support condition in (13) grows approximately with $\sqrt{n}$, while the strict minority condition reflects the assumed honest majority. This is a population-based support rule, not an estimate of the attacker share. For a given signal, write $Q_q(\mathcal{U})$ for its empirical quantile and $I_U = Q_{.75}(\mathcal{U}) - Q_{.25}(\mathcal{U})$ for its interquartile range (IQR). Low raw update norms require both

$$\begin{aligned} Q_{.25}(\mathcal{V}) - Q_{.75}(\mathcal{U}) &> I_U, \\ \min(\mathcal{V}) - Q_{.75}(\mathcal{U}) &> I_U. \end{aligned} \tag{14}$$

Equation (14) requires interquartile and full-peer separation. The individual norm must also lie below the peer median.

For high raw profile distances, let $I_V = Q_{.75}(\mathcal{V}) - Q_{.25}(\mathcal{V})$ and $g = Q_{.25}(\mathcal{U}) - Q_{.75}(\mathcal{V})$. Require $g > I_U$, together with

$$Q_{.25}(\mathcal{U}) - \max(\mathcal{V}) > I_U \quad \text{or} \quad \frac{I_U}{|\mathcal{U}| - 1} < \frac{I_V}{|\mathcal{V}| - 1}. \tag{15}$$

Equation (15) permits full-peer separation or greater within-cohort concentration. The individual's profile distance must exceed the peer median. Constant populations, invalid measurements or insufficient cohort support do not produce a statistical certificate.

The norm test requires the suspicious cohort to remain separated from the nearest peer, making overlap sufficient to reject certification. The profile test permits either full separation or a more concentrated cohort after the interquartile gap is established. Dividing each IQR by population size minus one compares typical spacing rather than spread alone. This allowance limits the influence of extreme peer distances, but can also admit persistent honest profile outliers. It is a geometric heuristic, not a calibrated false-positive test.

The same component must certify a client on both traps. The diagonal score nominates clients but cannot itself certify removal. Paired send-back supplies a separate evidence component without the statistical cohort requirement. Each client receives one of three cycle outcomes, strong, unresolved or clear. A nomination that fails certification remains unresolved without automatic quarantine. The collection of paired decisions is $\mathcal{E}$. These are cycle outcomes. A client's retained status may still reflect its recovery history.

C. Phases III and IV: Confirmation, Review and Audits

Separate counts $M_{i,n}$ and $M_{i,p}$ retain norm and profile certificates. Only a fresh paired certificate increments its own component, at most once per complete cycle. An ambiguous

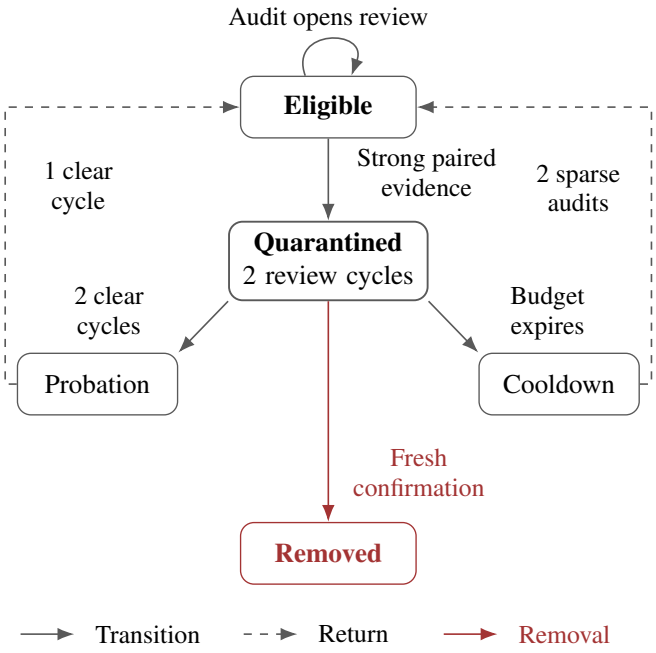


Fig. 4. Client participation and review. Fresh matching confirmation is tested for all non-removed clients, including after quarantine expires. The red arrow does not restrict removal to quarantined clients.

cycle preserves these counts; two consecutive complete clear cycles reset them. Removal requires a fresh certificate whose own count reaches two, even if the two certificates are not consecutive. Mixed components and retained memory alone cannot confirm removal. Exact current/previous sendback instead requires two consecutive complete paired cycles. Under this schedule, earliest removal is round 30.

Strong statistical evidence opens two cycles of quarantined review. Fresh strong evidence can renew it, but weak labels or old memory cannot. If it does not recur, exclusion expires even when the client remains unresolved. Two complete clear cycles rehabilitate a client, followed by one additional clear cycle of monitoring. Quarantine and removal are therefore different from a retained unresolved label. Fresh confirmation can remove a client after quarantine has expired, as Figure 4 summarizes.

Without active review or probation, ordinary training continues. After ten quiet rounds, one fresh challenge audits active clients. A single audit can open bounded review, but cannot confirm removal or clear paired certificate memory. Released clients wait through two audits before re-enrollment. The eligibility check uses cooldown before its decrement. The published engine restarts the quiet clock only for an empty raw finding set; the prospective review engine uses the corrected eligible-finding guard. These guards agree on all 622 recorded audits in the main study. Supplementary Section S5 documents that trace check and the liveness edge case.

D. Choice of Protocol Settings

Table I lists the settings shared by every reported configuration. Ten groups and a ten-round cycle provide a simple coverage schedule. Two challenges and two confirming cycles are the smallest repeated-evidence design used here. Review

Algorithm 1 Training with paired confirmation and bounded review

Initialize model, client records, evidence counts and quiet clock
for each round up to T **do**
  if warmup is incomplete **then**
    Send the ordinary model; aggregate all active responses
  else if initial coverage, active review or probation requires a cycle **then**
    On cycle start, freeze challenges and preselect response slots
    Collect responses; retain selected A/B summaries and sendback checks
    $\mathcal{A}_t \leftarrow$ ordinary, non-quarantined clients at round start
    if complete paired coverage is available **then**
      Score peers using (10)–(11)
      Apply support and certificates (13)–(15)
      Update same-component memory and bounded review
      Remove only on fresh matching confirmation
      Release expired quarantine; clear memory after two clear cycles
      Exclude new quarantines and removals from $\mathcal{A}_t$
      Set quiet clock if no review/probation remains, otherwise unset it
    end if
    Aggregate $\mathcal{A}_t$ using (12)
  else if a sparse audit is due **then**
    Test one challenge; open review only for eligible findings
    Restart quiet clock if no review/probation was opened
    Withhold challenge responses and keep the global model unchanged
  else
    Train and aggregate eligible ordinary responses
  end if
end for

and cooldown lengths bound exclusion and immediate re-enrollment, while the additional clear cycle extends monitoring before sparse auditing resumes. These are design tradeoffs, not statistically optimal choices.

Perturbations are scaled to each tensor so that a common relative strength can be used across layers. The scale floor handles tensors with negligible variance. Bootstrap replicates assess split stability, and the nominal confidence level controls the width of those split intervals. It does not supply a false-removal probability. The defaults were adopted during development and then held fixed across the reported grid. There is no separately held-out tuning study or sensitivity experiment, so the results do not establish an optimal parameter setting.

V. Results and Discussion

A. Experimental Setup

The study covers MNIST and CIFAR-10, IID and Dirichlet non-IID partitions, FR1–FR4, and free-rider proportions of 30% and 40% [28], [29]. Each configuration is evaluated across three random seeds, giving 96 detection runs in total. The honest-client impact and resource tables average the completed replications, while detection rates pool the removal counts. The optimizer and detector settings are held fixed.

Every federation has 100 clients and runs for 100 communication rounds. IID data are shuffled into equal-size client partitions. Non-IID label proportions follow a Dirichlet distribution with $\alpha = 0.5$, with allocation repeated if any client receives no samples. Partitioning, attacker identities,

initialization, local training and audit scheduling vary through separate random streams.

Honest clients perform three local SGD epochs, using learning rate 0.01, momentum 0.9 and batch size 32. The MNIST network has convolutional blocks with 16 and 32 channels and a 128-unit hidden classifier. CIFAR-10 uses 32, 64 and 128 channels with BatchNorm and a 256-unit hidden classifier. Images are normalized without augmentation. Execution uses the CPU, with six PyTorch threads. The evaluated FR3 history length is $H = 5$.

Two comparisons serve different purposes. Completed, seed-matched no-defense runs assess the effect of the whole BaitFL workflow. Separately instrumented BaitFL and STD-DAGMM executions measure timing stages over the full horizon. The latter are timing cases, separate from the multi-seed detection averages. STD-DAGMM is used for timing only. PASS, FRAD and FRIDA enter the requirements comparison, not a matched detection benchmark.

B. Evaluation Metrics

The evaluation asks whether BaitFL identifies free riders, preserves useful participation and adds little computational work. We assess these questions through detection performance, model accuracy, honest-client impact and resource use.

Detection performance. Precision measures the proportion of permanently removed clients that are free riders, while recall measures the proportion of free riders removed. We also report F_1 and the round at which removal of all free riders is complete. An unresolved label or temporary quarantine does not count as detection. If any free rider remains at the training horizon, complete detection is recorded as not reached.

Model accuracy and honest participation. Global test accuracy measures the utility of the trained model. Comparisons with no-defense federated averaging report accuracy differences in percentage points. To assess the cost to honest participants, we record permanent removals, clients ever quarantined and clients still unresolved at completion. Client-round counts measure how long responses are withheld. Total withheld participation includes trap responses and quarantine, counting each affected client-round once.

Lightweightness. Mean round time, execution duration and peak process resident memory (RSS) describe the cost of the complete simulation. The round timer covers training, response processing, detection, aggregation and global evaluation, ending before log and checkpoint publication. Execution duration also includes setup and publication work, but excludes powered-off waiting. Separate instrumented runs divide the work into computation stages and distinguish these from elapsed round time, which also includes logging and other execution overhead. Timing means cover all 100 rounds, spreading periodic decision costs over the full training horizon. Percentage comparisons use the corresponding no-defense or STD-DAGMM execution as the reference. Network transmission is not measured in this local simulator.

Each configuration has three complete runs. We treat the run as the experimental unit and report accuracy variation across those runs. Pooled precision summarizes the observed client outcomes, not uncertainty from independent client trials. Clients in a federation share a training trajectory, so the pooled percentage is not a confidence bound.

C. Permanent Detection and Model Utility

Table II summarizes the multi-seed results. Detection rates pool the removal counts across the four attacks and three seeds within each dataset, partition and attacker proportion. Final model accuracy is reported separately for FR1–FR4 as mean $\pm$ sample standard deviation.

In the main 30%/40% grid, all free riders are removed across 96 runs, giving 100.00% recall. There are 3 honest removals among 6,240 honest client-cases, concentrated in 2 runs. Pooled precision is 99.91% and F_1 is 99.96%, as descriptive summaries of these observed outcomes. Most executions complete detection at round 30. The slower cases extend to round 90, showing that successful detection need not occur at the first confirmation opportunity. Mean final test accuracy spans 98.83–99.02% on MNIST and 74.51–77.84% on CIFAR-10. Honest removals occur in two runs; per-run precision ranges from 95.24% to 100%. The pooled rate therefore summarizes errors concentrated in particular configurations rather than uniform behavior across runs. For MNIST non-IID at 40%, the 12 runs contain 480 correct removals and three honest removals, giving $480/483 = 99.38\%$ pooled precision.

FR1 and FR2 are consistently removed at the earliest confirmation round. FR1 returns an unchanged model and activates the exact-sendback path. FR2 avoids exact equality, but its fabricated noise remains distinct under the tested settings. The history-based attacks, FR3 and FR4, account for the delayed cases.

D. Honest-Client Impact

The principal errors occur among honest clients with non-IID data. Table III reports 3 permanent honest removals and 32 honest client-cases experiencing quarantine, totaling 620 quarantine client-rounds. No client remains quarantined at completion. Counts pool client-cases within each setting across FR1–FR4 and the study replications. Withholding counts traps and quarantine once per active honest client-round. Supplementary Table S2 retains the complete attack-specific means, ranges and unresolved-client counts.

The decision records identify the immediate causes of the false positives. Two honest clients in MNIST non-IID FR3 receive profile-distance certificates in successive cycles and are removed at round 30. A third honest client, in FR4, receives norm certificates in separated cycles and is removed at round 90. None of these removals follows exact sendback. Non-IID training can produce legitimate updates whose size or distribution across layers differs persistently from the population. When these updates also satisfy the geometric gate, repeated confirmation can reinforce the same mistaken suspicion. Fresh traps do not remove the underlying similarity, because a client's local data remain unchanged. This is a limitation of inferring training behavior from response statistics.

Unresolved labels can persist after the free riders have gone. For non-IID FR3 at 40%, the mean numbers of honest clients still unresolved at completion are 37 on MNIST and 43 on

TABLE II
DETECTION PERFORMANCE AND FINAL MODEL ACCURACY.

Dataset	Partition	FR (%)	Detection (%)		Final test accuracy (%)			
			Precision	Recall	FR1	FR2	FR3	FR4
MNIST	IID	30	100.00	100.00	99.01 ± 0.04	99.01 ± 0.04	99.02 ± 0.02	98.90 ± 0.07
		40	100.00	100.00	98.93 ± 0.06	98.94 ± 0.05	98.92 ± 0.07	98.83 ± 0.07
	Non-IID	30	100.00	100.00	98.92 ± 0.08	98.93 ± 0.06	98.93 ± 0.05	98.91 ± 0.05
		40	99.38	100.00	98.91 ± 0.07	98.91 ± 0.06	98.90 ± 0.05	98.88 ± 0.09
CIFAR-10	IID	30	100.00	100.00	77.84 ± 0.53	77.84 ± 0.31	77.76 ± 0.46	77.77 ± 0.37
		40	100.00	100.00	77.07 ± 0.55	77.13 ± 0.53	76.92 ± 0.52	76.97 ± 0.46
	Non-IID	30	100.00	100.00	75.94 ± 0.33	75.86 ± 0.65	75.77 ± 0.52	75.78 ± 0.36
		40	100.00	100.00	74.70 ± 1.22	74.88 ± 1.09	74.67 ± 0.64	74.51 ± 1.16

CIFAR-10, respectively 61.7% and 71.7% of the 60 honest clients. Unresolved denotes remaining response evidence rather than a free-rider classification. Ordinary responses from these clients remain eligible for aggregation, and they remain in the same-trap reference population. The label alone neither renews quarantine nor initiates dense probing, which depend on an active review budget or recovery probation. Such widespread unresolved labels have little value as an end-of-run classifier of free riding in these settings. Most withheld honest responses instead arise from the planned trap schedule. Even IID settings with no honest removal or quarantine withhold roughly 11% of honest responses. These forgone contributions are a utility cost even when the client is never removed.

E. What the Score Plots Establish

The round-20 examples in Figure 3b and Supplementary Figures S1–S2 show clear norm-axis separation in all displayed seed-42 panels, including FR3 and FR4. Those panels do not demonstrate overlap or establish separation in other cycles and seeds. The averages also do not replace the individual paired certificates.

Figure 5 instead examines a recorded error case: MNIST non-IID FR3 at 40%, with the two honest clients removed at round 30 marked explicitly. At round 30, the transformed paired-mean norm ranges partly overlap, unlike the representative seed-42 panels. The two removed honest clients receive repeated profile certificates. Because either statistical channel can certify a client, useful norm contrast elsewhere does not protect an honest client from the profile branch. These are averaged score plots, not substitutes for checking each challenge.

The tested fabrications also have favorable magnitude structure. FR1 has zero delta, FR2 uses small fixed noise, and FR3–FR4 recycle shared model changes. Averaging does not mathematically guarantee a smaller norm in every trajectory, but the displayed cases are strongly low-norm. Their results therefore cannot establish robustness to a norm-matched adversary or isolate the profile score's incremental value.

F. Oracle Norm-Matching Feature Diagnostic

To isolate that limitation, we replay each initial FR2–FR4 pair from the 96-run study: 72 cases across both datasets, partitions, proportions and the three study seeds. The unchanged detector must reproduce every stored round-20 certificate before stress testing. We then replace each free rider's norm by the same-trap median honest norm. Positive rescaling preserves the normalized profile in (9); profiles are held fixed and the peer scores and certificates are recomputed.

Evaluation labels construct this oracle stress input; they are not given to the detector. This provides the adversary extra information and operates on stored features rather than newly fabricated full models. It tests statistical certificates at one complete cycle, not confirmed removal, adaptation or model utility. FR1 is excluded because a zero vector cannot be positively rescaled to a nonzero norm. The low-norm channel certifies no attackers after norm matching. Profile certificates retain 61.9% coverage for FR2, but only 10.7% for FR3 and 17.9% for FR4, with ten honest certificates across the stress cases. Profiles therefore supply some magnitude-independent evidence, while leaving a substantial detection gap. Whether these certificates would recur and confirm removal is unobserved here. Both channels retain the joint nomination step. These are certificate breakdowns within the unchanged pipeline, not comparisons of separate norm-only and profile-only detectors.

G. Honest-Only and Lower-Prevalence Checks

The prospective review engine freezes the v15.4 statistical settings and corrects only the audit-clock liveness guard. Four honest-only 100-round runs cover both datasets and both partitions with seed 42. None removes an honest client. This is evidence from four federations, not a population false-positive guarantee.

Table V reports the complete non-IID 5%, 10% and 20% grid available at this revision. FR1 removes all free riders at each share. FR2–FR4 remove none at 5%, while the completed 10% and 20% cases remove all. The cohort-support requirement in (13) is ten at 100 active clients, making a five-client statistical cohort ineligible. This limit is retained rather than relaxed after observing misses. CIFAR-10 FR3 at 10% also removes one honest client.

At the evidence cutoff, 26 of 48 low-prevalence cases are complete, including two IID MNIST 5% cases where FR3 and FR4 remain undetected. Passive ablation/MAD diagnostics, oracle utility comparisons, published decaying-noise and scaled-delta tests, and the registered unseen-seed phase are pending. They are not reported as completed evidence.

H. Execution Time and Memory

Table VI reports multi-seed process measurements averaged over the attacks, with minimum and maximum execution values

TABLE III
HONEST-CLIENT COSTS IN THE MAIN STUDY.

Dataset	Partition	FR (%)	Honest cases	Removals	Ever quarantined	Withheld (%)
MNIST	IID	30	840	0	0	11.00
MNIST	IID	40	720	0	0	11.17
MNIST	Non-IID	30	840	0	4	11.52
MNIST	Non-IID	40	720	3	13	12.90
CIFAR-10	IID	30	840	0	0	11.00
CIFAR-10	IID	40	720	0	0	11.00
CIFAR-10	Non-IID	30	840	0	4	12.45
CIFAR-10	Non-IID	40	720	0	11	12.69

Counts pool completed client-cases across FR1–FR4 and study replications. A client can recur across runs. Withholding is the fraction of active honest client-rounds affected by traps or quarantine, counted once. Full attack-level means and ranges are in Supplementary Table S2.

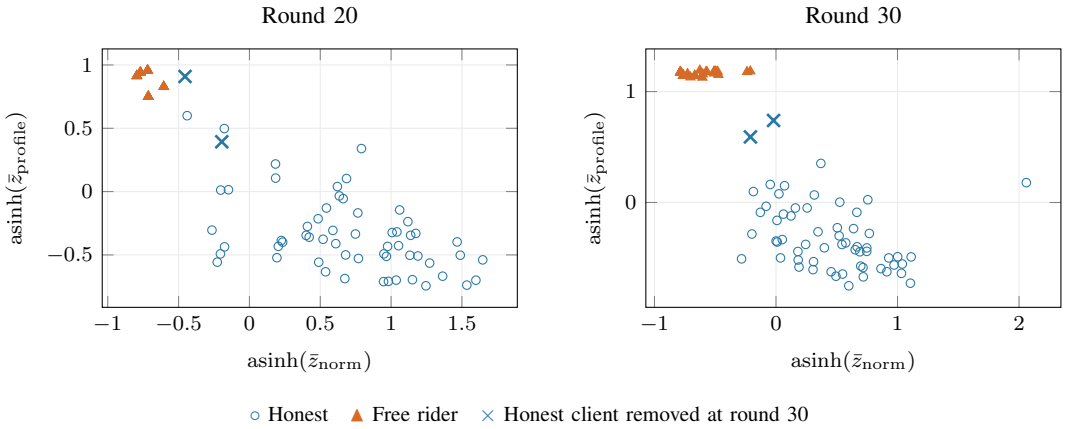


Fig. 5. A recorded honest-client error case. MNIST non-IID FR3, 40% free riders, study seed 43. Each point averages selected A/B scores. Crosses mark the two honest clients removed by repeated profile certificates. Points can coincide.

TABLE IV
ROUND-20 STATISTICAL CERTIFICATES UNDER ORACLE NORM MATCHING.

	Stored responses: attacker certificates (%)			Norm matched		
Attack	Norm	Profile	Either	Norm (%)	Profile/either (%)	Honest certificates
FR2	100.0	100.0	100.0	0.0	61.9	5
FR3	38.1	50.0	79.8	0.0	10.7	2
FR4	78.6	34.5	82.1	0.0	17.9	3

Each attack row covers 24 federations, 840 attacker client-cases and 1,560 honest client-cases across both datasets, partitions, 30%/40% and three seeds. Rates pool attacker certificates; honest entries are counts. Norm matching uses an oracle honest median and keeps scale-free profiles fixed. The endpoint is one paired statistical certificate, not removal.

TABLE V
COMPLETED NON-IID LOWER-PREVALENCE CHECKS.

Dataset	FR (%)	FR1	FR2	FR3	FR4
MNIST	5	5/5	0/5	0/5	0/5
MNIST	10	10/10	10/10	10/10	10/10
MNIST	20	20/20	20/20	20/20	20/20
CIFAR-10	5	5/5	0/5	0/5	0/5
CIFAR-10	10	10/10	10/10	10/10†	10/10
CIFAR-10	20	20/20	20/20	20/20	20/20

Entries are free riders permanently removed / present by round 100. One frozen seed, 100 clients per run. † One honest removal; all other displayed runs have none.

in brackets. Runs at 30% free riders include more honest training jobs than those at 40%, which helps explain their longer rounds. Process RSS includes training tensors, simulated attacker histories and server state. It should not be interpreted as detector-only memory.

The overall cost and the cost of the checking stages answer different questions. Table VI describes the complete simulation. Section V-J measures selected operations, while Section VI analyzes their scaling.

I. Comparison with No-Defense Federated Averaging

The no-defense baseline keeps all clients throughout training and aggregates every response. It performs no traps, screening, quarantine or removal. Each completed baseline is matched to BaitFL using the same dataset, partition, attack, proportion and random seed. Configuration checks and identical warmup accuracies verify that the paired runs start from the same training conditions.

Table VII uses 81 completed matched runs: 33 on MNIST and 48 on CIFAR-10. Means are calculated across matched seeds within each attack, then across the four attacks with equal weight. A BaitFL execution without a completed baseline partner does not enter this comparison, even though it remains in Table II. Positive accuracy changes favor BaitFL. A positive time change means longer rounds. Accuracy differences are

TABLE VI
EXECUTION TIME AND PROCESS MEMORY.

Dataset	Partition	FR (%)	Round time (s) mean [range]	Peak process RSS (GiB) mean [range]	Execution duration (min/run)
MNIST	IID	30	42.38 [39.78–48.57]	0.54 [0.29–0.78]	75.02
		40	37.05 [35.35–42.68]	0.60 [0.30–0.88]	66.71
	Non-IID	30	41.62 [40.57–43.43]	0.57 [0.30–0.78]	73.79
		40	36.08 [34.34–38.67]	0.62 [0.30–0.88]	65.27
CIFAR-10	IID	30	99.35 [88.18–119.07]	1.24 [0.70–1.76]	175.07
		40	85.35 [73.52–104.21]	1.38 [0.71–2.06]	154.65
	Non-IID	30	92.30 [82.85–108.38]	1.31 [0.73–1.79]	161.97
		40	77.03 [69.10–84.85]	1.49 [0.75–2.14]	139.40

expressed in percentage points.

The available paired means show comparable or slightly higher final accuracy with BaitFL. Their descriptive mean differences are 0.08 percentage points on MNIST and 0.61 on CIFAR-10, with variation across attacks and partitions. MNIST baseline coverage remains uneven, while CIFAR-10 now covers the complete matched grid. These differences do not establish a systematic accuracy gain across random seeds. That assessment requires paired differences with uncertainty estimates from more complete replication coverage. MNIST rounds become modestly shorter, while the current CIFAR-10 comparisons take somewhat longer than no defense. These are whole-workflow measurements of the full training procedure. Stage-level timing is examined separately below.

The observed accuracy differences describe the combined effect of detection and mitigation. Removing fabricated responses changes aggregation, but challenge rounds also withhold genuine updates. Final model utility therefore depends on both effects, rather than on removal recall alone.

Figure 6 shows model accuracy from rounds 20–100, averaging all three BaitFL replications and the completed no-defense runs. Baseline coverage is 2–3 replications per attack setting on MNIST and 3 on CIFAR-10. FR1–FR4 receive equal weight, and each dataset row shares an accuracy scale. The curves show how learning continues after initial challenge coverage. Coverage differs between methods, so the paired accuracy differences in Table VII remain the basis for the quantitative comparison.

Figure 7 uses saved paired trajectories to compare logged time to 98% accuracy on MNIST and 70% on CIFAR-10. These illustrative targets were chosen for this analysis and reached in every included execution. Times sum the round timers through the first crossing, then average matched seeds within attack and the four attacks equally. Coverage is 33 MNIST pairs and 48 CIFAR-10 pairs. The comparison combines free-rider removal with withheld training contributions. Removing fabricated responses can reduce update dilution, so shorter times do not isolate audit costs. Separating these effects requires an honest-only oracle baseline without audits. Timer boundaries and execution load also differ, as in Table VII. These measurements describe the saved executions and do not establish a controlled time-to-accuracy advantage.

J. Instrumented Timing and Lightweight Operation

Dedicated executions compare BaitFL with a reimplementation of STD-DAGMM on non-IID FR3 at 40%, using both datasets, 100 clients, 100 rounds, three local epochs and six CPU threads. STD-DAGMM freshly fits its auxiliary model for five epochs per round and retains every client. That fitting budget is a declared reproduction choice, not an author-specified training schedule or evidence of convergence [6], [10]. The instrumented BaitFL run matches its corresponding study accuracy trajectory in every round.

Table VIII separates training from response preparation, challenge construction, decisions, aggregation and evaluation. BaitFL's recorded challenge, feature/preparation and decision stages average 215.21 ms per round on MNIST and 791.03 ms on CIFAR-10. They account for 0.58% and 0.89% of the corresponding computation totals. STD-DAGMM adds auxiliary fitting, averaging 15.76 and 42.09 seconds. The checking subtotal groups these stages. The final rows give BaitFL's absolute saving in seconds per round together with the percentage reduction relative to STD-DAGMM. Stage times include sample standard deviations across the 100 rounds, capturing variation between ordinary training and periodic decisions.

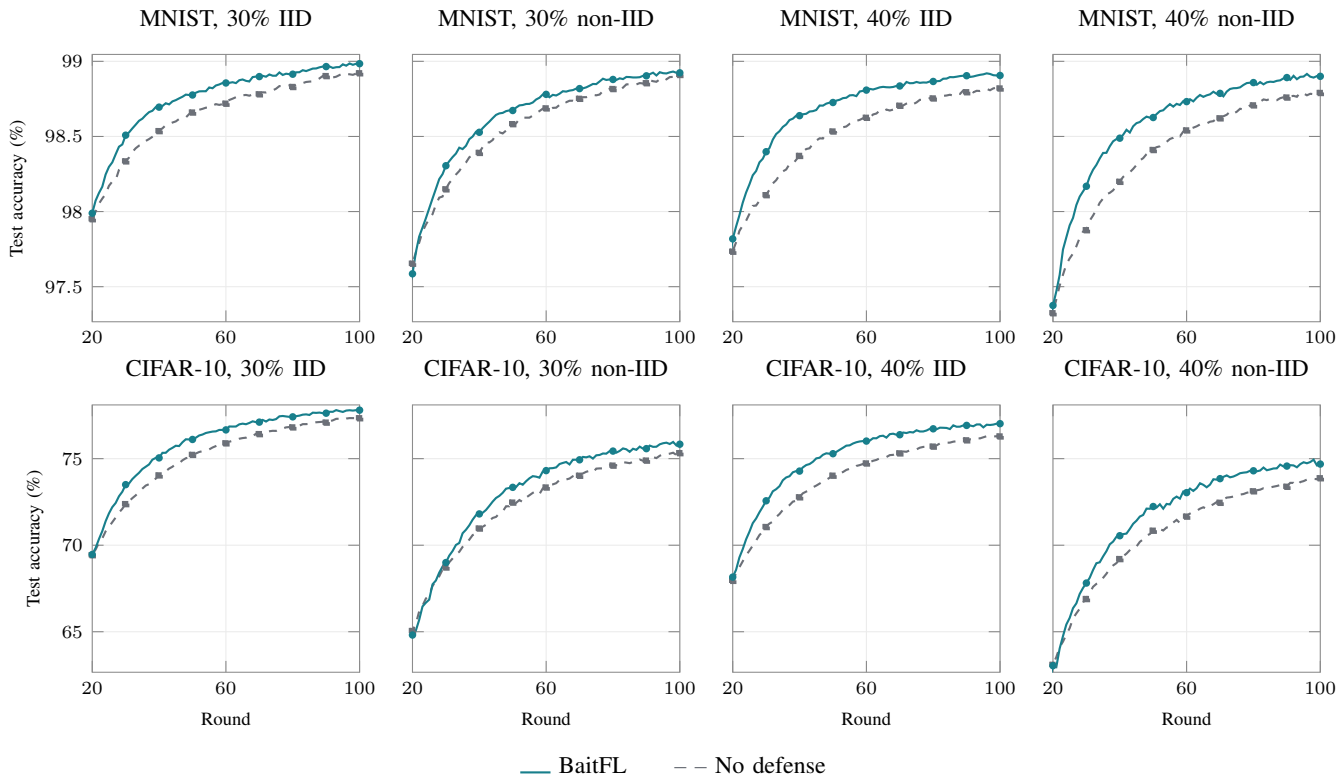
Figure 8 shows average computation and elapsed round time over the complete horizon. BaitFL uses 29.35% less computation time on MNIST and 26.23% less on CIFAR-10 in these executions. Elapsed-time reductions are 34.87% and 27.20%. Logging, checkpoints and other runtime costs contribute to the difference between computation and elapsed time. Network communication is absent from both measurements.

These timing cases describe different native workflows. BaitFL changes probing and participation, whereas STD-DAGMM keeps the full population. Removing free riders does not remove honest clients' local SGD jobs. Checking accounts for less than 1% of measured computation, while the STD-DAGMM comparison measures a particular fitting budget. Detailed per-round and fitting-stage records accompany the study.

The stored STD-DAGMM flags also permit a limited diagnostic of this five-epoch recipe. Each timing execution contains 4,000 free-rider responses, none of which is flagged. The numbers of flagged honest responses are 2,200 on MNIST and 2,196 on CIFAR-10, giving zero pooled flag precision and recall in both executions. These are roundwise flags from one configuration and fitting budget. A comparative detection benchmark would require matched decision endpoints and baseline settings, so these observations do not establish a ranking against BaitFL's permanent-removal results.

TABLE VII
COMPARISON WITH NO-DEFENSE AGGREGATION.

Dataset	Partition	FR (%)	Final accuracy (%)			Mean round time (s)		
			No defense	BaitFL	Difference (percentage points)	No defense	BaitFL	Change (%)
MNIST	IID	30	98.92	98.99	+0.07	44.49	41.64	-6.39
		40	98.82	98.91	+0.09	38.10	36.96	-3.01
	Non-IID	30	98.91	98.94	+0.03	45.46	41.44	-8.84
		40	98.79	98.93	+0.14	37.35	35.50	-4.96
CIFAR-10	IID	30	77.37	77.80	+0.44	94.90	99.35	+4.68
		40	76.31	77.02	+0.71	81.28	85.35	+5.01
	Non-IID	30	75.35	75.84	+0.49	91.28	92.30	+1.12
		40	73.90	74.69	+0.79	74.52	77.03	+3.37



BaitFL: three replications per setting. No defense: 2–3 on MNIST and 3 on CIFAR-10. Curves average replications within attack, then weight FR1–FR4 equally.

Fig. 6. Mean global test accuracy during training.

The unassigned-runtime entry is the difference between elapsed time and the sum of explicitly timed computation and I/O stages. Its larger value for STD-DAGMM shows that the two execution paths include different untimed work. It is excluded from computation totals and cannot be attributed to detector fitting. Elapsed-time savings include this difference, so they are not an isolated measure of detection overhead.

Low checking cost does not imply immediate detection. Figure 9 shows the variation in complete-removal rounds across configurations. Markers indicate the mean completion round, with whiskers spanning the observed minimum and maximum. The wider ranges in history-based attacks reflect the additional challenge cycles sometimes needed for confirmation.

VI. COMPLEXITY AND LIGHTWEIGHT DESIGN

Let N be the largest active population, P the floating model coordinates, L the floating tensors, B bootstrap replicates and $K = \lceil \sqrt{N} \rceil$ bins. Feature extraction reads $O(NP)$ coordinates per round. Leave-one-out profile comparisons cost $O(N^2 L)$, with $O(N^2 \log N)$ additional scalar-median work in the current implementation. Bootstrap splitting costs $O(B(N+K))$ on scalars. For T rounds, M paired cycles and A sparse audits, total incremental detector work is

$$\mathcal{W}_{\text{det}} = O\big(TNP + (M+A)\big[N^2(L+\log N) + B(N+K)\big]\big). \tag{16}$$

Equation (16) excludes ordinary training, aggregation and persistent logging. The detector is linear in model size for feature extraction and quadratic in client count for peer comparisons; it is not claimed to scale linearly in the federation

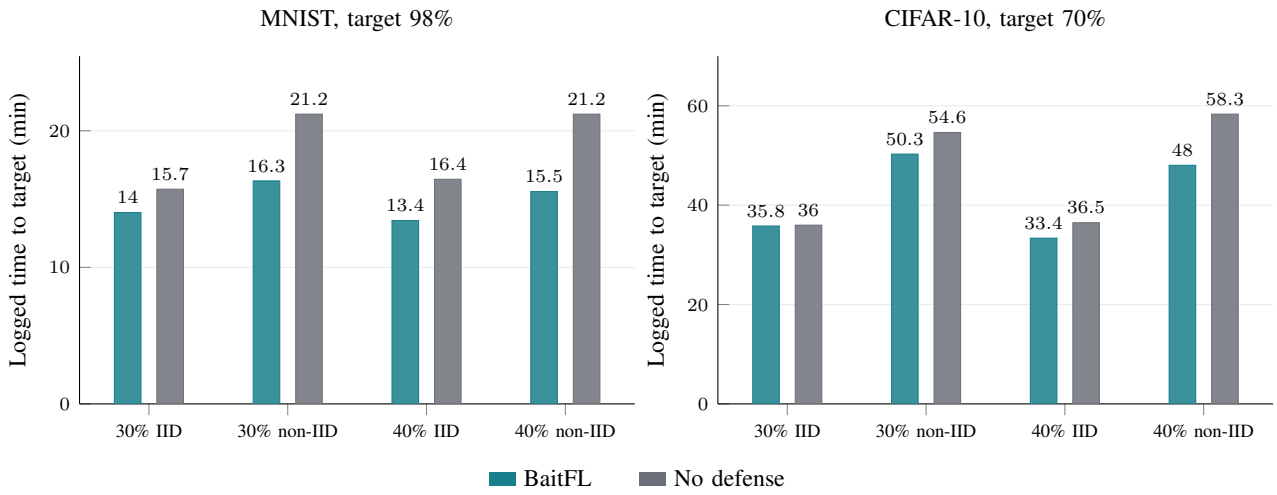


Fig. 7. Time to first target-accuracy crossing in matched runs. BaitFL is the left bar and no defense the right bar in each pair.

TABLE VIII
ROUND-TIME BREAKDOWN.

Stage	MNIST		CIFAR-10	
	BaitFL	STD-DAGMM	BaitFL	STD-DAGMM
Honest local training	34.30 ± 0.94	33.89 ± 0.66	81.37 ± 5.40	72.42 ± 3.68
Fabricated responses	0.018 ± 0.029	0.066 ± 0.011	0.188 ± 0.172	0.224 ± 0.022
Client diagnostic evaluation	0.161 ± 0.248	0	0.899 ± 0.752	0
Traps and scheduling	0.0018 ± 0.0055	0	0.018 ± 0.058	0
Response features and preparation	0.198 ± 0.050	0.045 ± 0.003	0.729 ± 0.179	0.115 ± 0.012
Auxiliary detector fitting	0	15.76 ± 1.25	0	42.09 ± 1.87
Scoring and state decisions	0.016 ± 0.071	0.335 ± 0.039	0.043 ± 0.145	0.827 ± 0.057
Aggregation	0.030 ± 0.013	0.043 ± 0.003	0.096 ± 0.039	0.136 ± 0.014
Global evaluation	2.20 ± 0.10	2.13 ± 0.04	5.21 ± 0.44	4.24 ± 0.20
Logging and checkpoints	3.99 ± 0.61	3.75 ± 0.54	11.34 ± 1.79	9.76 ± 1.30
Unassigned runtime	0.0088 ± 0.0026	6.82 ± 2.33	0.054 ± 0.044	7.49 ± 2.08
Checking subtotal (included above)	0.215 ± 0.093	16.14 ± 1.25	0.791 ± 0.244	43.03 ± 1.91
Checking share of computation (%)	0.58	30.89	0.89	35.84
BaitFL time saved relative to STD-DAGMM				
Computation saved: s/round (%)	15.34 (29.35%)		31.50 (26.23%)	
Elapsed saved: s/round (%)	21.91 (34.87%)		37.35 (27.20%)	

Times are mean $\pm$ sample SD in seconds per round over 100 rounds. Combined stages are summed per round before calculating SD. Checking subtotals are included in the preceding stages. Shares and savings use complete-horizon means, relative to STD-DAGMM for savings.

population.

Working state is $O(P + NL + BN + BK)$, plus $O(N)$ evidence counters and fingerprints. At 100 clients, packed paired summaries occupy about 8.59 KiB on MNIST and 19.53 KiB on CIFAR-10. Shared float32 challenges occupy about 1.58 and 4.74 MiB. Python objects, temporary arrays, client models and attacker histories make full-process RSS larger. Supplementary Tables S3–S4 give operation-level bounds and payload calculations.

Challenges replace ordinary model requests, so the nominal payload order remains $O(NP)$ per round. Withholding genuine responses can change time to target accuracy. No network or energy saving follows from unchanged payload order. The measured sub-1% checking fractions support lightweight server computation only for the tested population and architectures.

VII. Discussion and Limitations

A. Information Available to the Server

BaitFL uses returned models, client and challenge identifiers, response norms, profile distances and exact-sendback checks. Attacker identities and behavior settings belong to the simulator and evaluation records. They do not enter screening, geometric tests or confirmation. Likewise, the configured attacker proportion does not set a removal quota. This separation prevents direct use of evaluation labels, although the method's design has been informed by experimental observations.

B. Confirmation and Honest-Client Safety

Statistical removal requires fresh evidence from the same component on both selected challenges and in another complete cycle. Mixed norm/profile findings cannot be combined to reach confirmation, and stored evidence alone cannot authorize removal. Paired exact sendback also requires repetition. Weak unresolved labels do not renew quarantine, and unconfirmed review expires when fresh strong evidence does not recur.

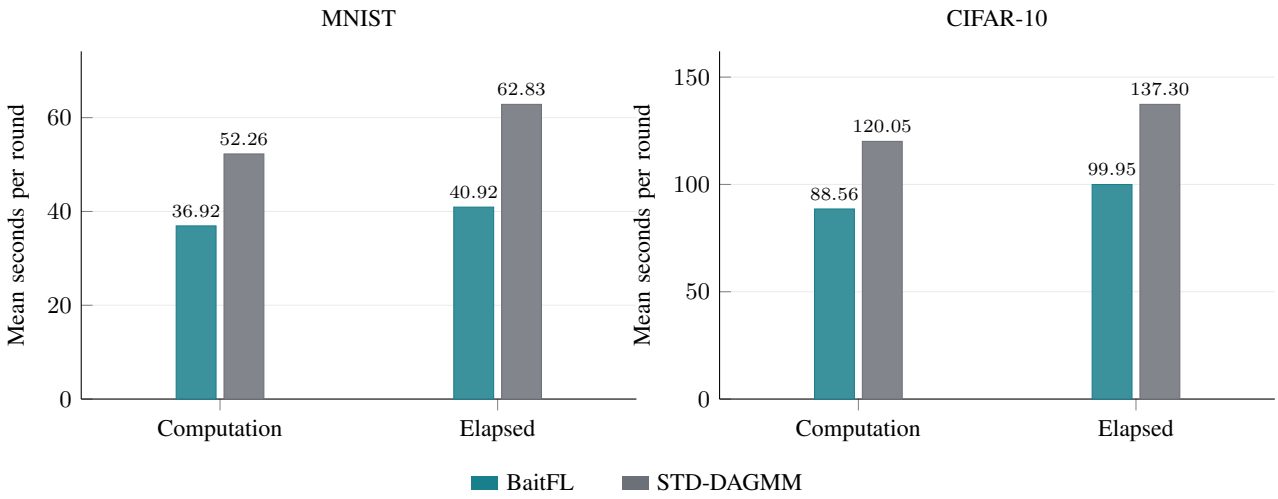


Fig. 8. Average computation and elapsed time per round.

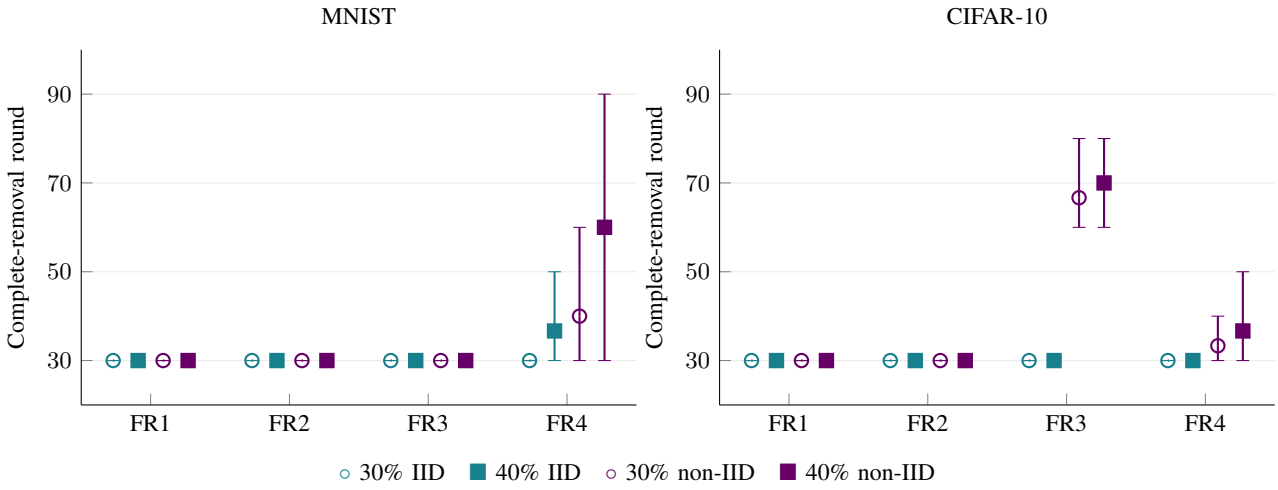


Fig. 9. Free-rider removal completion rounds.

These rules reduce reliance on isolated anomalies. They do not make repeated evidence statistically independent. An honest client keeps its local data across challenges and can therefore remain unusual in successive cycles. Certificate memory can accumulate such findings unless two complete clear cycles reset it, explaining the risk of late false removal observed in the experiments.

The decision boundaries are derived from the observed population rather than an attack-fitted numerical z-score cutoff. Nevertheless, the method has design parameters, including challenge strength, bootstrap confidence, cohort support and confirmation lengths. At 100 clients, statistical certification requires a cohort of at least ten. A small group of free riders that avoids exact sendback may therefore remain uncertified.

C. Adaptive Attacks and Identifiability

A response-only detector cannot distinguish two clients whose observable histories are identical, even if one trained and the other fabricated its responses. BaitFL therefore tests whether challenges expose differences in behavior. Its certificates do not prove that local computation occurred.

The evaluated free riders are not told which models are challenges. An adaptive participant might still recognize repeated frozen models, unusual perturbations or differences revealed through communication with other clients. Reproducible simulation randomness also does not establish unpredictability against an attacker who knows its configuration. Deployment would require server-secret randomness and tests against challenge-aware strategies. Secrecy alone would not guarantee that challenge models are indistinguishable from ordinary ones.

D. Privacy and Participation Limits

The server inspects individual returned models, so the current protocol cannot be used directly with standard secure aggregation [13]. It does not access raw client data or require reported training loss, but these restrictions do not provide differential privacy or prevent information leakage through updates [2]–[4].

The reference population assumes an honest majority. Colluding clients could share trained responses, while Sybil identities could distort the reference distribution. Neither case is evaluated. The experiments also assume complete participation. Incomplete challenge coverage cannot certify removal, and the simulator stops on a nonfinite response rather than providing a full malicious-message or dropout-handling

protocol. Checkpoint recovery restores the recorded training and evidence state after an interruption. Authentication and availability defenses require additional mechanisms.

E. Evaluation Scope and Remaining Tests

The main grid covers three seeds, 100 clients, one architecture per dataset and one non-IID concentration. The new lower-prevalence checks use one frozen seed and expose misses at 5%; they do not support a prevalence- independent recall claim. The norm-matching feature diagnostic isolates statistical behavior but is not a full adaptive attack. Component and passive-detector ablations, completed published-attack runs and a stronger detection baseline are still needed. The stored five-epoch STD-DAGMM recipe supplies flag diagnostics, not a fair ranking of detection methods.

Future tests should vary local training, non-IID concentration, model architecture, BatchNorm-buffer treatment, collusion and participation. LEAF and FedScale provide relevant heterogeneous settings [30], [31]. The held-out seed phase is registered but unexecuted. Oracle utility comparisons are also pending, so the benefit of removing fabricated responses is not yet separated from the cost of withholding honest challenge responses. Larger-scale detector memory, communication and energy measurements remain necessary.

F. Code and Evidence Availability

The project repository is available at

https://github.com/Aashnan-Rahman/Mitigating-Free-Rider

The study retains configurations, per-client decisions, per-round timing and source hashes for reproducibility. A public immutable archive of the exact manuscript evidence is not yet available.

VIII. Conclusion

BaitFL combines model challenges, compact response statistics and repeated fresh evidence. The 96-run 30%/40% grid removes all tested free riders, with three honest removals in two runs. Four additional honest-only runs have no honest removal, but the completed non-IID 5% tests miss FR2–FR4. These outcomes delimit the support gate's operating range rather than establish universal recall. The norm-matching diagnostic further tests what magnitude contrast alone cannot establish. Checking remains below 1% of computation in the two instrumented workflows. Reliable auditing under lower prevalence, stronger fabrication and heterogeneous honest behavior remains the central research problem.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[2] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021. [Online]. Available: https://arxiv.org/abs/1912.04977

[3] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/60a6c4002cc7b29142def8871531281a-Abstract.html

[4] L. Melis, C. Song, E. D. Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *IEEE Symposium on Security and Privacy*, 2019, pp. 691–706. [Online]. Available: https://doi.org/10.1109/SP.2019.00029

[5] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 108. PMLR, 2020, pp. 2938–2948. [Online]. Available: https://proceedings.mlr.press/v108/bagdasaryan20a.html

[6] J. Lin, M. Du, and J. Liu, "Free-riders in federated learning: Attacks and defenses," arXiv preprint arXiv:1911.12560, 2019. [Online]. Available: https://arxiv.org/abs/1911.12560

[7] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 130. PMLR, 2021, pp. 1846–1854. [Online]. Available: https://proceedings.mlr.press/v130/fraboni21a.html

[8] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, "PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2024. [Online]. Available: https://doi.org/10.1109/JIOT.2023.3288936

[9] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, "FRIDA: Free-rider detection using privacy attacks," *Journal of Information Security and Applications*, vol. 97, 2026, art. no. 104357. [Online]. Available: https://doi.org/10.1016/j.jisa.2025.104357

[10] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=BJJLHbb0-

[11] Z. Zhu, J. Shu, X. Zou, and X. Jia, "Advanced free-rider attacks in federated learning," in *NeurIPS Workshop on New Frontiers in Federated Learning*, 2021. [Online]. Available: https://neurips2021workshopfl.github.io/NFFL-2021/papers/2021/Zhu2021.pdf

[12] B. Wang, H. Li, X. Liu, and Y. Guo, "FRAD: Free-rider attacks detection mechanism for federated learning in AIoT," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024. [Online]. Available: https://doi.org/10.1109/JIOT.2023.3298606

[13] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191. [Online]. Available: https://research.google/pubs/practical-secure-aggregation-for-privacy-preserving-machine-learning/

[14] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/6617-machine-learning-with-adversaries-byzantine-tolerant-gradient-descent.pdf

[15] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 5650–5659. [Online]. Available: https://proceedings.mlr.press/v80/yin18a.html

[16] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Transactions on Signal Processing*, vol. 70, pp. 1142–1154, 2022. [Online]. Available: https://doi.org/10.1109/TSP.2022.3153135

[17] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Network and Distributed System Security Symposium*, 2021. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/ndss2021_6C-2_24434_paper.pdf

[18] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, vol. 2, 2020. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html

[19] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 5132–5143. [Online]. Available: https://proceedings.mlr.press/v119/karimireddy20a.html

[20] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://papers.neurips.cc/paper_files/paper/2020/hash/564127c03caab942e503ee6f810f54fd-Abstract.html

[21] H. Jia, M. Yaghini, C. A. Choquette-Choo, N. Dullerud, A. Thudi, V. Chandrasekaran, and N. Papernot, "Proof-of-learning: Definitions and practice," in *IEEE Symposium on Security and Privacy*, 2021, pp. 1039–1056. [Online]. Available: https://arxiv.org/abs/2103.05633

[22] L. Zhao, Q. Wang, C. Wang, Q. Li, C. Shen, and B. Feng, "VeriML: Enabling integrity assurances and fair payments for machine learning as a service," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 10, pp. 2524–2540, Oct. 2021. [Online]. Available: https://doi.org/10.1109/TPDS.2021.3068195

[23] A. Ghorbani and J. Zou, "Data shapley: Equitable valuation of data for machine learning," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 2242–2251. [Online]. Available: https://proceedings.mlr.press/v97/ghorbani19c.html

[24] R. Jia, D. Dao, B. Wang, F. A. Hubis, N. Hynes, N. M. Gurel, B. Li, C. Zhang, D. Song, and C. Spanos, "Towards efficient data valuation based on the shapley value," in *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 89, 2019, pp. 1167–1176. [Online]. Available: https://proceedings.mlr.press/v89/jia19a.html

[25] Z. Liu, Y. Chen, H. Yu, Y. Liu, and L. Cui, "GTG-Shapley: Efficient and accurate participant contribution evaluation in federated learning," *ACM Transactions on Intelligent Systems and Technology*, vol. 13, no. 4, 2022, art. no. 60. [Online]. Available: https://arxiv.org/abs/2109.02053

[26] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. [Online]. Available: https://doi.org/10.1109/TSMC.1979.4310076

[27] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

[28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://doi.org/10.1109/5.726791

[29] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html

[30] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečný, H. B. McMahan, V. Smith, and A. Talwalkar, "LEAF: A benchmark for federated settings," arXiv preprint arXiv:1812.01097, 2018. [Online]. Available: https://arxiv.org/abs/1812.01097

[31] F. Lai, Y. Dai, S. Singapuram, J. Liu, X. Zhu, H. Madhyastha, and M. Chowdhury, "FedScale: Benchmarking model and system performance of federated learning at scale," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 11 814–11 827. [Online]. Available: https://proceedings.mlr.press/v162/lai22a.html